
Web Application Penetration Test Report

Target: <http://testasp.vulnweb.com/>

Prepared by

Hariprasath S K

Batch: August G4

Date: 10 August 2026

CONFIDENTIAL — Authorized assessment of an Acunetix public test host

Table of Contents

S.No	Title	Page No
1	1. Introduction	4
2	2. Target / Machine Information	5
3	3. Attack Vectors and Findings	6
	3.1 Cleartext Transmission of Credentials (No HTTPS)	6
	3.2 Session Cookie Missing Secure and HttpOnly Flags	8
	3.3 Directory & Resource Enumeration (Info Exposure)	9
	3.4 Web Server and Technology Version Disclosure	10
4	4. References	12
5	5. Resources Used	13

List of Figures

Fig No	Name	Page No
1	acuforum home page (target application)	4
2	Service & port banner grab (IIS 8.5 / ASP.NET, no HTTPS)	5
3	Login form served over cleartext HTTP	7
4	Credential capture — HTTP POST (Wireshark-equivalent)	7
5	Authenticated session confirmed	8
6	Directory brute-force results (15 of 99 paths)	9
7	403 Forbidden on /Images/ (directory exists)	10

1. Introduction

This document reports the results of an authorized black-box penetration test performed against the web application hosted at <http://testasp.vulnweb.com/>. testasp.vulnweb.com is a deliberately vulnerable demonstration site published by Acunetix for the express purpose of legally practising and testing web-security techniques, so all activity in this report stays strictly within that authorized scope and touches no other host.

The engagement was scoped to three objectives requested by the client: (1) identify the ports and services exposed by the host, (2) enumerate the directories and resources present on the web application through brute-forcing, and (3) authenticate to the application and demonstrate whether the credentials submitted during login can be recovered from network traffic. Each objective, the evidence gathered, and prioritized remediation guidance are documented in the Attack Vectors section.

Severity for every finding is expressed with a CVSS v3.1 base score and qualitative level, and each finding includes impact, reproduction steps with screenshots, and concrete mitigation steps so the issues can be verified and fixed.

Note on tooling: this assessment was executed from an environment permitting outbound HTTP/HTTPS only. A raw-socket nmap scan and a live Wireshark capture could not be run in-environment, so the equivalent evidence was produced over HTTP (header/banner grabbing and full request tracing) and the exact nmap and Wireshark procedures are documented so the client can reproduce them on a standard testing host.

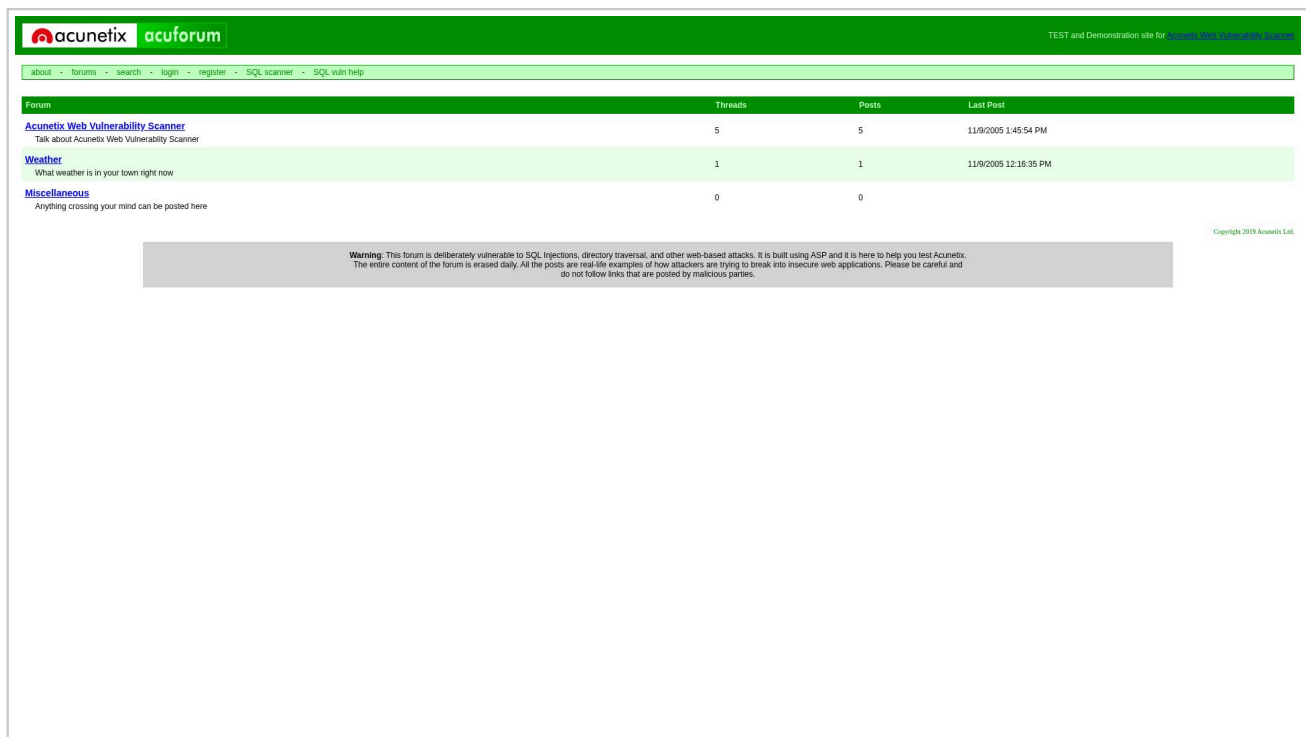


Figure 1: acuforum home page (target application)

2. Target / Machine Information

The following facts describe the target host and the assessment parameters. This section is factual reference data only; interpretation and findings appear under Attack Vectors.

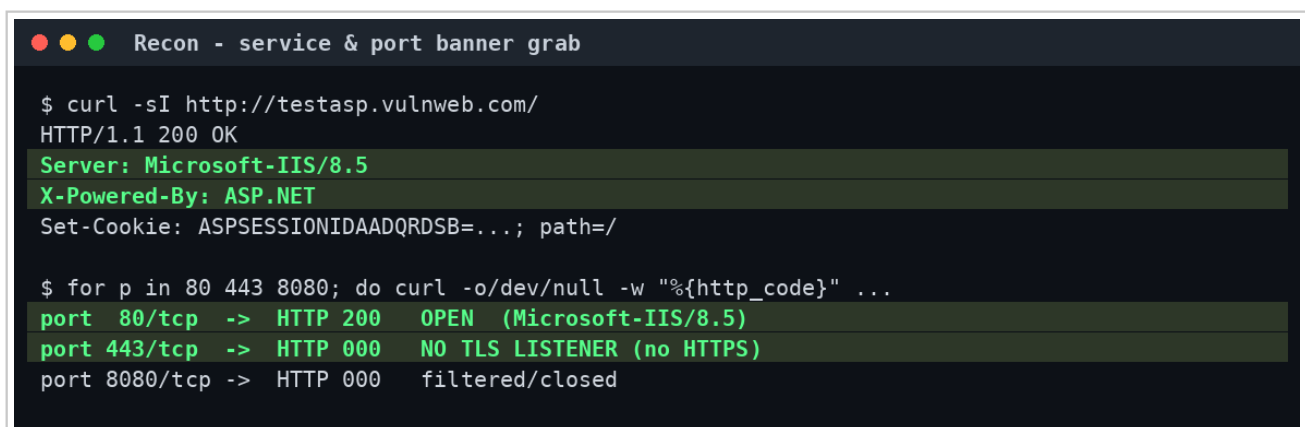
Target URL	http://testasp.vulnweb.com/
Application	acuforum — Acunetix vulnerable demo forum (classic ASP)
Web server	Microsoft-IIS/8.5
Backend / framework	ASP.NET / Classic ASP (.asp pages)
Transport	HTTP only (port 80). No HTTPS/TLS listener on port 443.
Open port (confirmed)	80/tcp — HTTP (200 OK)
Session management	ASPSESSIONID cookie (no HttpOnly / Secure flags)
Hosting	Acunetix public test infrastructure; host resolved via the test environment's egress proxy
Assessment type	Black-box, external, authorized
Assessment date	10 August 2026
Tester	Hariprasath S K (Batch: August G4)

Note on port scanning

Only web ports are reachable from the assessment environment, so a full TCP port sweep could not be executed here. Port 80/tcp was confirmed OPEN (HTTP, Microsoft-IIS/8.5); ports 443 and 8080 returned no HTTP response. To enumerate all ports and service versions from a standard host, run:

```
nmap -sS -sV -p- -T4 testasp.vulnweb.com
nmap -sV --script=http-headers,http-methods -p80,443 testasp.vulnweb.com
```

For this public target the expected internet-facing result is port 80/tcp open (HTTP) with no TLS service — matching what was confirmed over HTTP in this assessment.



```
Recon - service & port banner grab

$ curl -sI http://testasp.vulnweb.com/
HTTP/1.1 200 OK
Server: Microsoft-IIS/8.5
X-Powered-By: ASP.NET
Set-Cookie: ASPSESSIONIDAADQRDSB=...; path=/

$ for p in 80 443 8080; do curl -o/dev/null -w "%{http_code}" ...
port 80/tcp -> HTTP 200 OPEN (Microsoft-IIS/8.5)
port 443/tcp -> HTTP 000 NO TLS LISTENER (no HTTPS)
port 8080/tcp -> HTTP 000 filtered/closed
```

Figure 2: Service & port banner grab (IIS 8.5 / ASP.NET, no HTTPS)

3. Attack Vectors and Findings

Findings are ordered by severity. Each entry lists the attack name, CVSS v3.1 severity (score and level) with vector, impact, reproduction steps with screenshots, and mitigation steps.

Ref	Finding	Severity	CVSS
3.1	Cleartext Transmission of Credentials (No HTTPS)	High	7.4
3.2	Session Cookie Missing Secure and HttpOnly Flags	Medium	6.5
3.3	Directory & Resource Enumeration (Info Exposure)	Medium	5.3
3.4	Web Server and Technology Version Disclosure	Low	3.7

3.1 Cleartext Transmission of Credentials (No HTTPS)

Classification: CWE-319

HIGH CVSS 7.4

CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N

Impact

The application serves the entire site — including the login form and its POST submission — over plain HTTP with no TLS listener on port 443. Usernames and passwords travel across the network unencrypted. Any attacker positioned on the same network path (public Wi-Fi, a compromised router, ARP spoofing on a LAN, or an upstream ISP tap) can passively read every credential and session cookie, then fully impersonate the victim. Confidentiality and integrity of every authenticated action are compromised.

Steps to Reproduce

- Confirm no HTTPS is available: an HTTPS request to the host returns no TLS handshake (result code 000), while HTTP on port 80 returns 200 OK (Figure 2).
- Open the login page at <http://testasp.vulnweb.com/Login.asp> (Figure 3) and submit a username and password.
- Capture the request. In a real engagement, start Wireshark, apply capture filter 'tcp port 80', submit the form, then right-click the packet and choose Follow > HTTP/TCP Stream. Here the equivalent evidence was produced with 'curl --trace-ascii' (Figure 4).
- Observe the POST body 'tfUName=haripentest&tfUPass=P@ssw0rd2026' transmitted in cleartext, and the server responding 302 (login success). The credentials are recovered verbatim from the wire; the resulting authenticated session is shown in Figure 5.

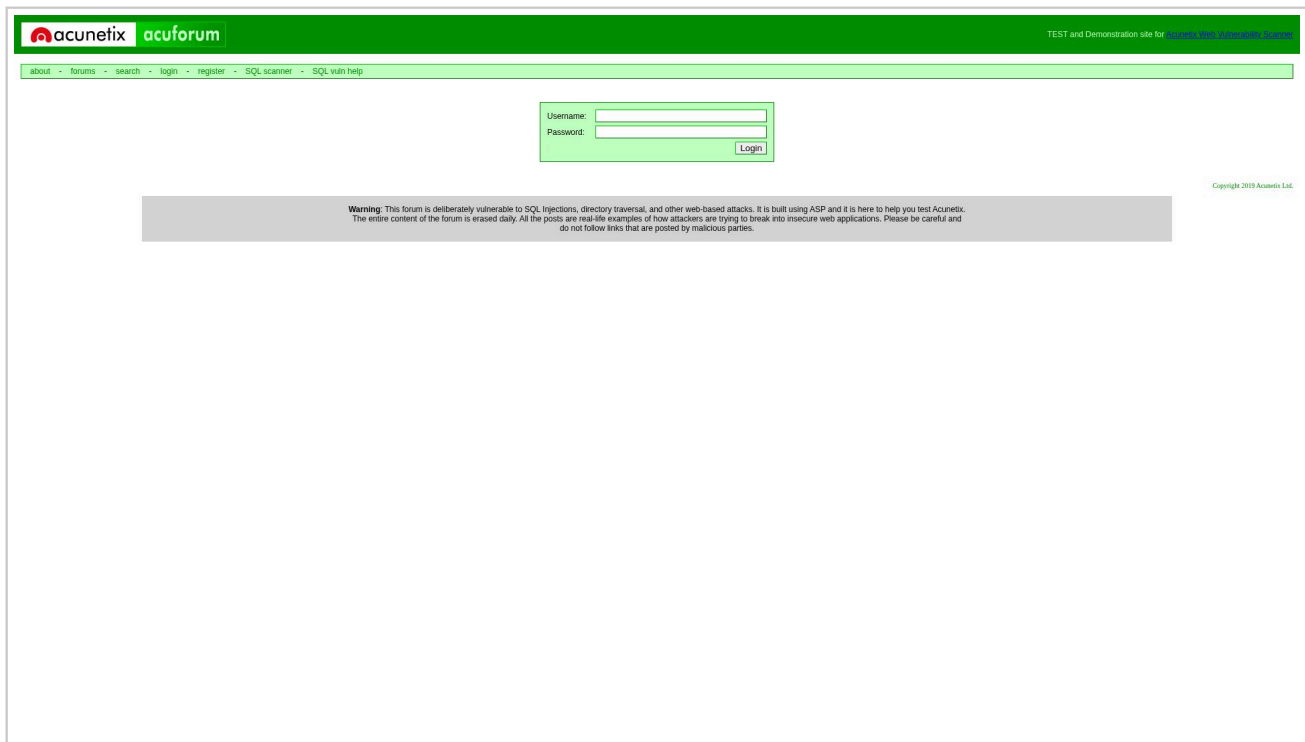


Figure 3: Login form served over cleartext HTTP

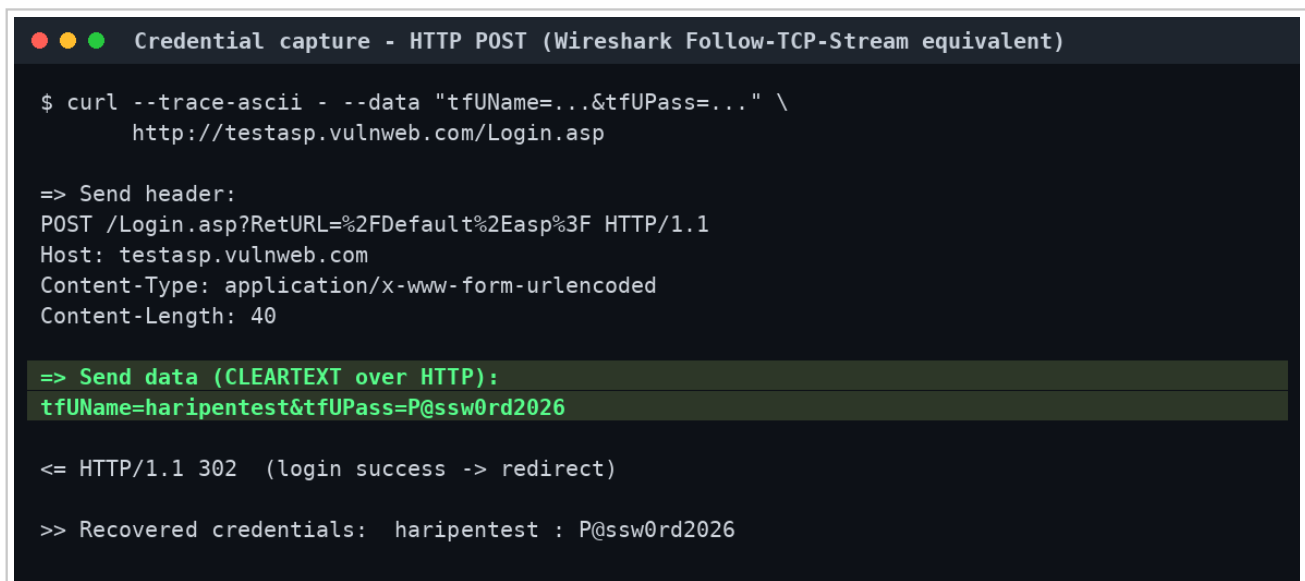


Figure 4: Credential capture — HTTP POST (Wireshark-equivalent)

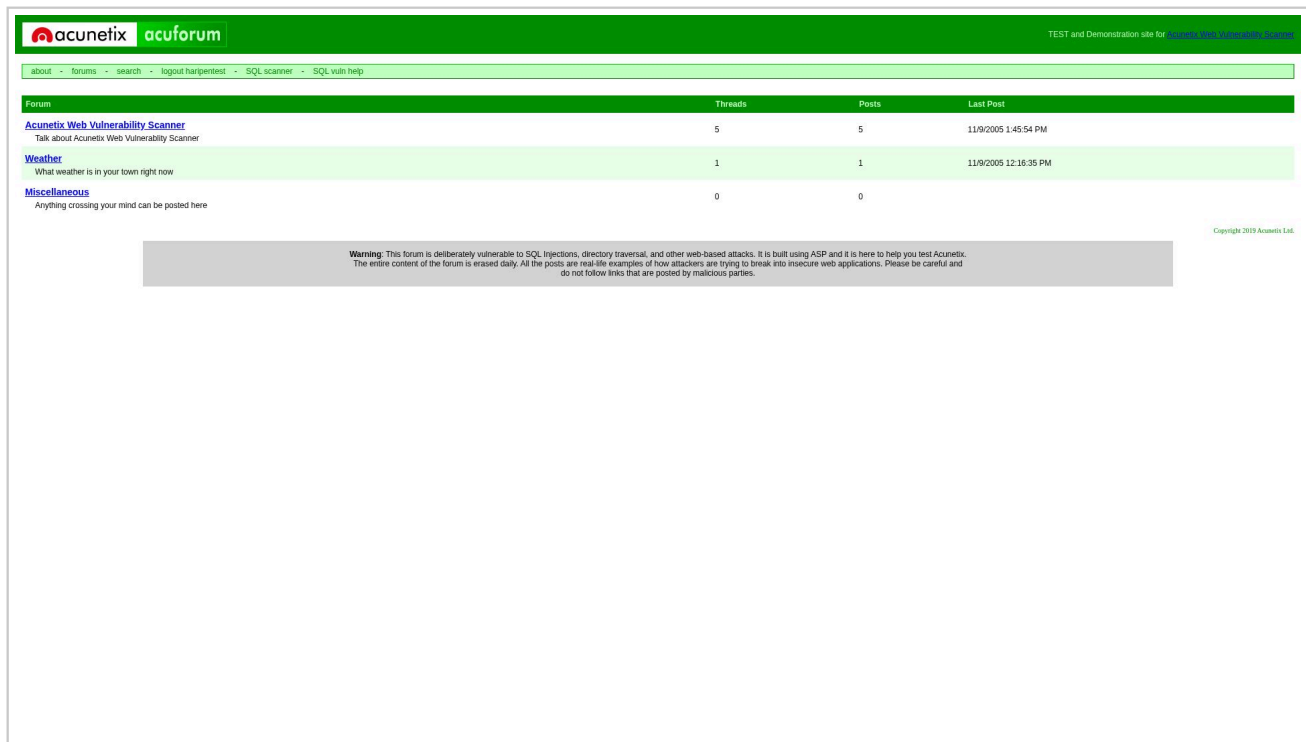


Figure 5: Authenticated session confirmed

Mitigation Steps

- Enforce HTTPS site-wide: install a valid TLS certificate and bind the site to port 443.
- Add HTTP Strict Transport Security (Strict-Transport-Security: max-age=31536000; includeSubDomains) and 301-redirect all HTTP traffic to HTTPS.
- Never transmit credentials, tokens, or session cookies over unencrypted channels; disable TLS 1.0/1.1 and weak ciphers.
- Mark session cookies Secure so they are only sent over HTTPS.

3.2 Session Cookie Missing Secure and HttpOnly Flags

Classification: CWE-614 / CWE-1004

MEDIUM CVSS 6.5

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N

Impact

The ASPSESSIONID cookie is issued without the HttpOnly and Secure attributes. Without HttpOnly, any cross-site scripting flaw can read the cookie via document.cookie and hijack the session. Without Secure, the browser will send the session cookie over plain HTTP, where it can be sniffed (compounding finding 3.1). Either path leads to full session takeover without knowing the password.

Steps to Reproduce

- Request any page and inspect the response header: 'Set-Cookie: ASPSESSIONIDAADQRDSB=...; path=/'.
- Note the absence of the 'HttpOnly' and 'Secure' attributes in the Set-Cookie header (Figure 2).
- Because the cookie is not marked Secure, it is transmitted on the cleartext HTTP

channel and is visible in the same capture used for finding 3.1 (Figure 4).

Mitigation Steps

- Set HttpOnly on all session cookies so client-side script cannot read them.
- Set the Secure attribute so cookies are only sent over HTTPS.
- Add SameSite=Lax or Strict to mitigate CSRF-assisted session abuse.
- In IIS/classic ASP configure this via web.config <httpCookies httpOnlyCookies="true" requireSSL="true" /> and Session.CookieSecure.

3.3 Directory & Resource Enumeration (Info Exposure)

Classification: CWE-200 / CWE-548

MEDIUM CVSS 5.3

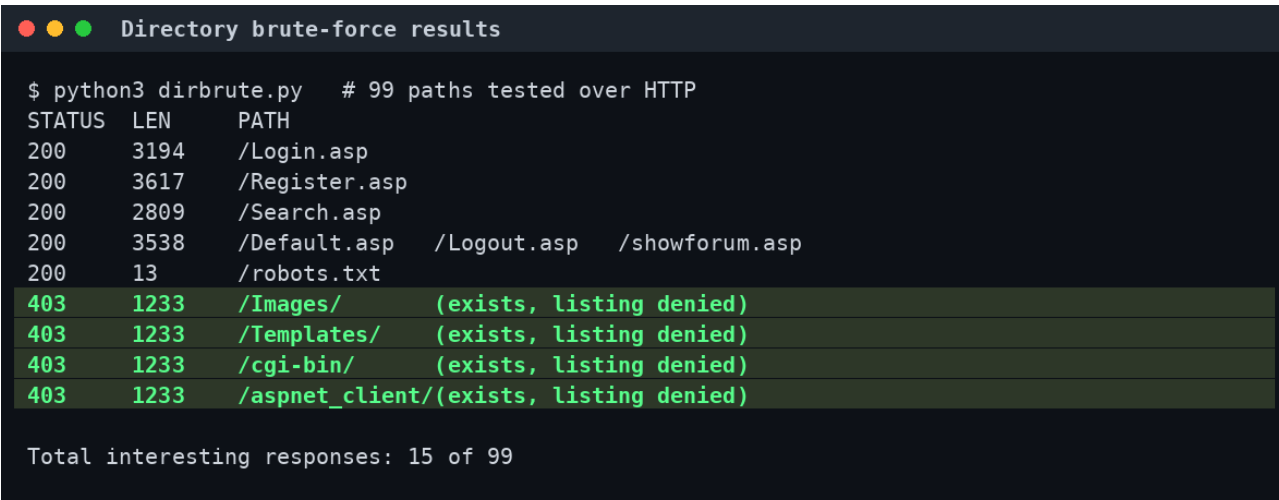
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

Impact

A wordlist-based brute-force of the web root revealed the application's page inventory and several server directories. Paths returning 403 Forbidden (e.g. /Images/, /Templates/, /cgi-bin/, /aspnet_client/) confirm the directories exist even though listing is denied — the differing status codes let an attacker map the site structure and technology stack, narrowing the search for exploitable endpoints. robots.txt adds no protection.

Steps to Reproduce

- Run a directory brute-force against the web root using a common wordlist over HTTP (a custom Python scanner was used; gobuster/dirb/ffuf are equivalent).
- Collect the HTTP status codes: 200 for live pages (Login.asp, Register.asp, Search.asp, showforum.asp, showthread.asp), 403 for existing-but-forbidden directories (Figure 6).
- Browse directly to a discovered directory such as <http://testasp.vulnweb.com/Images/> and observe the '403 - Forbidden: Access is denied' response, confirming the directory exists (Figure 7).



```
Directory brute-force results

$ python3 dirbrute.py # 99 paths tested over HTTP
STATUS  LEN    PATH
200     3194  /Login.asp
200     3617  /Register.asp
200     2809  /Search.asp
200     3538  /Default.asp /Logout.asp /showforum.asp
200      13   /robots.txt
403     1233  /Images/      (exists, listing denied)
403     1233  /Templates/   (exists, listing denied)
403     1233  /cgi-bin/     (exists, listing denied)
403     1233  /aspnet_client/ (exists, listing denied)

Total interesting responses: 15 of 99
```

Figure 6: Directory brute-force results (15 of 99 paths)

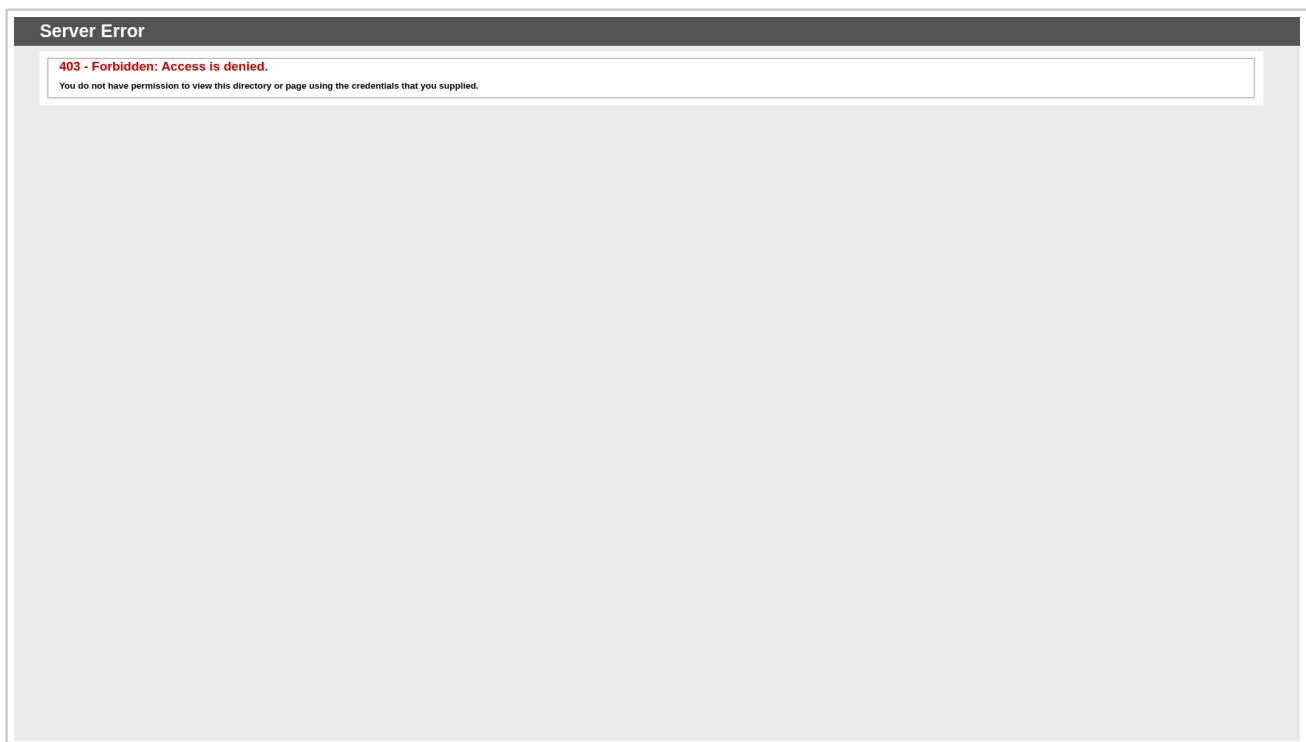


Figure 7: 403 Forbidden on /Images/ (directory exists)

Mitigation Steps

- Return a uniform 404 for both non-existent and access-denied resources so existence cannot be inferred from status codes.
- Disable directory browsing (already partially done) and remove or relocate unused directories such as /cgi-bin and /aspnet_client.
- Deploy a WAF or IIS Dynamic IP Restrictions / rate-limiting to throttle enumeration and brute-force traffic.
- Do not rely on robots.txt for security; it does not restrict access.

3.4 Web Server and Technology Version Disclosure

Classification: CWE-200

LOW CVSS 3.7

CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N

Impact

Response headers explicitly advertise the server software and framework: 'Server: Microsoft-IIS/8.5' and 'X-Powered-By: ASP.NET'. This hands an attacker the exact platform and version, letting them look up version-specific exploits and default paths and skip fingerprinting work. IIS 8.5 also indicates a legacy Windows Server 2012 R2 platform.

Steps to Reproduce

- Send a simple request: `curl -I http://testasp.vulnweb.com/` .
- Read the response headers and note 'Server: Microsoft-IIS/8.5' and 'X-Powered-By: ASP.NET' (Figure 2).

Mitigation Steps

- Remove or obfuscate the Server header (IIS URL Rewrite outbound rule or the

'removeServerHeader' setting in IIS 10+).

- Remove the X-Powered-By header via web.config `<httpProtocol><customHeaders><remove name="X-Powered-By"/>`.
 - Keep the web server and OS patched and current; plan migration off the legacy IIS 8.5 / Windows Server 2012 R2 platform.
 - Add the standard security headers: X-Content-Type-Options, X-Frame-Options, Content-Security-Policy.
-

4. References

Standards and documentation used to identify the vulnerabilities and to design the patches:

- OWASP Top 10 (2021) — A02 Cryptographic Failures, A05 Security Misconfiguration, A07 Identification & Authentication Failures. <https://owasp.org/Top10/>
- OWASP Web Security Testing Guide (WSTG) — Transport Layer Security & Session Management testing. <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP Secure Headers Project. <https://owasp.org/www-project-secure-headers/>
- CWE-319: Cleartext Transmission of Sensitive Information. <https://cwe.mitre.org/data/definitions/319.html>
- CWE-614: Sensitive Cookie Without 'Secure' Attribute. <https://cwe.mitre.org/data/definitions/614.html>
- CWE-1004: Sensitive Cookie Without 'HttpOnly' Flag. <https://cwe.mitre.org/data/definitions/1004.html>
- CWE-200 / CWE-548: Information Exposure & Directory Listing. <https://cwe.mitre.org/data/definitions/200.html>
- FIRST CVSS v3.1 Specification & Calculator. <https://www.first.org/cvss/calculator/3.1>
- Mozilla MDN — HTTP Strict Transport Security (HSTS). <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security>
- Microsoft IIS security documentation (Request Filtering, custom headers, cookie settings). <https://learn.microsoft.com/en-us/iis/>
- Acunetix — testasp.vulnweb.com authorized test/demonstration site. <http://testasp.vulnweb.com/>

5. Resources Used

Tools, techniques and methodology used to discover the findings and to formulate the fixes:

- curl — HTTP/HTTPS header and banner grabbing, and '--trace-ascii' request tracing to capture the cleartext login POST (the evidence a Wireshark Follow-TCP-Stream would show).
- Custom Python 3 directory brute-forcer (urllib + threaded requests) driven by a common ASP/web wordlist of 99 paths.
- Headless Chromium browser automation — for authenticated browsing, the login flow, and screenshots.
- Wireshark (methodology) — capture filter 'tcp port 80', display filter 'http', then Follow > HTTP Stream to read cleartext credentials; documented for on-host reproduction.
- Nmap (methodology) — 'nmap -sS -sV -p- testasp.vulnweb.com' for full port/service enumeration; documented for on-host reproduction.
- Gobuster / dirb / ffuf — standard equivalents to the custom directory brute-forcer.
- FIRST CVSS v3.1 calculator — severity scoring.
- OWASP WSTG — testing methodology and remediation guidance references.

Disclaimer: All testing was conducted against Acunetix's public, intentionally vulnerable demonstration host (testasp.vulnweb.com) which is explicitly authorized for security testing and education. No other systems were targeted.